Universidad Nacional de Costa Rica

Sede Regional Chorotega

Campus Nicoya

Curso: Programación I EIF201

Profesor: Francisco Coulon

Nombre del Proyecto:

Proyecto BlackJack

Integrantes:

Gerson Gómez García

504720251

Kendal Obando Mayorga

504710364

Josué Alonso Díaz Calvo

504740617

Brandon Gutiérrez Centeno

504700897

Wender Villgegas García

504510755

Fecha:

30 de octubre de 2026

Introducción

El presente documento tiene como finalidad exponer los avances en el desarrollo de un proyecto de programación basado en el juego de cartas *Blackjack*, implementado mediante el lenguaje C++. Se presenta la estructura general del sistema a través de un diagrama UML, así como una explicación de los principales componentes del código desarrollados hasta el momento.

El proyecto se ha trabajado de manera progresiva, comenzando con el diseño y la definición de las clases que conforman el sistema, sus respectivos atributos, métodos y relaciones. Asimismo, se ha iniciado la implementación de las funcionalidades correspondientes a la administración de cartas, el cálculo de sus valores, la gestión de las apuestas y el control del dinero disponible. Para ello, se ha empleado el paradigma de programación orientada a objetos, organizando el código en archivos de encabezado (.h) y archivos de implementación (.cpp).

A lo largo del proceso, se ha buscado establecer una estructura modular que facilite la comprensión, el mantenimiento y la integración de los diferentes componentes del programa. Debido a que el proyecto continúa en desarrollo, algunas funcionalidades se encuentran pendientes de implementación o integración. Por consiguiente, este documento se enfoca en describir el diseño planteado, el trabajo realizado y los avances alcanzados hasta esta etapa, como base para la continuación y finalización del sistema.

Definición del problema

El profesor Francisco Coullon nos asignó el proyecto final de programación 1 el cual es realizar con el paradigma Programación Orientada a Objetos (POO), el famoso juego de mesa "Blackjack", nuestro reto también tendría que ver con descifrar el modo o el flujo de trabajo a seguir a la hora de hacer el proyecto.

El proyecto también requería varios módulos de funciones, como tener 6 mazos en zapato (osea 312 cartas), el juego es por turnos, se deben implementar características básicas como, pedir, plantarse, dividir y doblar, seguir las reglas del blackjack tradicional, administrar los pagos o cobros de dinero y validar cada una de las apuestas, y eso conlleva también repartirse el trabajo, trabajar en equipo y tener una comunicación asertiva, los integrantes debemos tener responsabilidad con el proyecto y responsabilidad afectiva al liderar o decir algún comentario.

La entrega consiste en 2 entrega, la fase 1 y la fase 2, este documento forma parte de la fase 1 de entrega.

Sumado a todo lo anterior, la plataforma por excelencia para subir y administrar un proyecto de esta índole es "GitHub", utilizando el IDE "Visual Studio" mediante "Git", para tener un flujo de proyecto ordenado por ramas y actualizaciones.

Reglas del juego

Para facilitar los cálculos, se supone que cada jugador comienza con una apuesta de 100 fichas, salvo cuando se indica una apuesta adicional.

Ejemplo 1

- **Apuesta inicial:** 100 fichas.
- **Jugador:** 10♠ + 8♥ = 18 puntos.
- **Crupier:** 9♣ + 7♦ = 16 puntos.
- **Acción:** el jugador se planta con 18. El crupier pide 5♣ y alcanza 21.

Resultado: el jugador pierde, porque 18 es menor que 21.

- Pago neto: −100 fichas.
- Total devuelto al jugador: 0 fichas

Ejemplo 2

- **Apuesta inicial:** 100 fichas.
- **Jugador:** 10♥ + 7♣ = 17 puntos.
- **Acción:** el jugador pide 8♦ y alcanza 25 puntos.
- **Crupier:** K♠ + 6♥ = 16 puntos; después pide 4♣ y termina con 20.

Resultado: el jugador pierde inmediatamente por superar 21.

- Pago neto: −100 fichas.
- Total devuelto al jugador: 0 fichas.

Ejemplo 3

- **Apuesta inicial:** 100 fichas.
- **Jugador:** K♠ + 7♥ = 17 puntos.
- **Crupier:** 10♣ + 7♦ = 17 puntos.
- **Acción:** ambos se plantan.

Resultado: las dos manos tienen 17 puntos y ninguna es blackjack.

- Pago neto: 0 fichas.
- Total devuelto al jugador: 100 fichas

Ejemplo 4

- **Apuesta inicial:** 100 fichas.
- **Jugador:** A♠ + K♥ = 21 puntos en dos cartas.
- **Crupier:** 9♦ + 7♣ = 16 puntos.
- **Acción:** el crupier pide 4♠ y termina con 20 puntos.

Resultado: el jugador gana con blackjack natural.

El pago es de 3:2:

- Ganancia: 100 fichas × 1,5 = 150 fichas.
- Devolución de la apuesta: 100 fichas.
- **Total, recibido: 250 fichas.**

Ejemplo 5

- **Apuesta inicial:** 100 fichas.
- **Mano original:** 8♥ + 8♣ = 16 puntos.
- **Acción:** el jugador divide la pareja y añade otra apuesta de 100 fichas.

Las manos resultantes son:

- **Mano A:** 8♥ + 3♠ = 11; pide K♦ y termina con 21.
- **Mano B:** 8♣ + 10♠ = 18; se planta.
- **Crupier:** 10♣ + 6♦ = 16; pide 4♥ y termina con 20.

Resultado:

- Mano A: gana con 21 frente a 20. Ganancia neta de 100 fichas.
- Mano B: pierde con 18 frente a 20. Pérdida de 100 fichas.
- **Balance neto total: 0 fichas**

Al ser dividido, el 21 de la mano A no es blackjack natural.

Ejemplo 6

- **Apuesta inicial:** 100 fichas.
- **Jugador:** 5♥ + 6♣ = 11 puntos.
- **Acción:** el jugador dobla, añade 100 fichas y recibe una sola carta.
- **Carta adicional:** K♦. El jugador termina con 21.
- **Crupier:** 10♠ + 6♦ = 16; pide 8♣ y se pasa con 24.

Resultado: el jugador gana porque el crupier supera 21.

- Apuesta total: 200 fichas.
- Ganancia neta: 200 fichas.
- Devolución de las apuestas más la ganancia: **400 fichas**.

Estos seis ejemplos cubren los resultados obligatorios y las acciones de dividir y doblar.

Diagrama de Clases UML

Responsabilidades de las clases

El sistema modela una partida de Blackjack entre hasta cinco jugadores y un crupier, dentro de un casino. Las clases se dividen en tres grupos: la mesa y sus participantes, las cartas, y las enumeraciones que describen estados y resultados.

Mesa y participantes

Casa

Representa el casino. Posee el dinero con el que se pagan las apuestas ganadas y fija los límites de apuesta de la mesa. Valida que una apuesta esté entre el mínimo y el máximo, entrega dinero cuando se paga a un jugador y lo recibe cuando se cobra una apuesta perdida.

Colabora con: Juego, Crupier.

Juego

Representa la partida de Blackjack y dirige la ronda completa: apuestas, reparto, turnos, resolución y cierre. Lleva el estado de la partida y el número de ronda, muestra las reglas y da el turno a cada participante. También baraja el mazo, compara a cada jugador con el crupier, y paga o cobra según el resultado. La partida termina cuando ningún jugador conserva dinero.

Colabora con: Jugador, Crupier, Mazo, Casa, EstadoJuego, ResultadoRonda.

Participante

Es la clase base de Jugador y Crupier. Reúne lo que tiene cualquiera sentado a la mesa: un nombre y una mano de cartas. Recibe cartas, muestra su mano y la limpia al terminar la ronda. Obliga a cada clase hija a decidir por sí misma si quiere otra carta.

Colabora con: Mano, Jugador, Crupier.

Jugador

Es quien apuesta. Administra sus fichas, aparta su apuesta, recibe fichas cuando gana y entrega las que pierde. Decide si pide carta o se planta consultando al usuario.

Colabora con: Participante, Ficha, Juego.

Crupier

Es el empleado de la casa que atiende la mesa. Reparte cartas a los jugadores y a sí mismo, acepta y valida las apuestas, destapa su carta oculta y recoge las cartas al terminar la ronda. Juega con una regla fija: pide carta mientras su mano valga menos de 17.

Colabora con: Participante, Casa, Mazo, Jugador.

Ficha

Representa una pieza de dinero. Solo guarda su valor y su color.

Colabora con: Jugador.

Cartas
Carta

Representa una carta de la baraja: su palo, su número y su valor en el juego. Sabe si es un As y cómo mostrarse. No sabe en qué mano o mazo se encuentra.

Colabora con: Mazo, Mano.

Mazo

Es el zapato del casino, formado por varias barajas. Crea sus cartas al construirse, entrega la carta de arriba, avisa cuando quedan pocas cartas y conviene barajar. No decide cuándo ni quién lo baraja.

Colabora con: Carta, Juego, Crupier.

Mano

Guarda las cartas que tiene un participante durante una ronda. Calcula su valor, considerando que el As cuenta 1 u 11, detecta el blackjack natural y si la mano se pasó de 21. Puede mostrarse ocultando una carta, como ocurre con la del crupier, y se vacía al terminar la ronda.

Colabora con: Participante, Carta.

Listas Enum:

EstadoJuego

Define las fases de la partida: MENU, APOSTANDO, TURNO_JUGADOR, TURNO_CRUPIER, PAGANDO y FIN_RONDA. Juego la usa para saber qué corresponde hacer en cada momento.

ResultadoRonda

Define el resultado de enfrentar a un jugador con el crupier: GANA, PIERDE, EMPATE o BLACKJACK. De este resultado depende cuánto se paga.

Plan de acción del proyecto

Fecha	Actividad	Responsables	Resultado o avance
20/10/2026	Planteamiento inicial del UML y creación de un repositorio Git de prueba para practicar commits, pull y push.	Equipo de trabajo.	Primer planteamiento UML y práctica inicial de Git.
21/10/2026	Revisión del UML con la tutora, aplicación de recomendaciones, creación del repositorio del proyecto y de la rama feature/esqueletoClases; elaboración de algunos archivos .h.	Equipo de trabajo; Gerson compartió conocimientos sobre Git.	Repositorio creado, rama inicial y archivos de encabezado.
22/10/2026	Discusión sobre la distribución del trabajo y finalización de los archivos .h.	Equipo de trabajo.	Estructura inicial de clases completada.

23/10/2026	Inicio de los archivos .cpp y posterior corrección de errores detectados en archivos .h; consulta de dudas al profesor y revisión del UML.	Equipo de trabajo.	Correcciones a los encabezados y observaciones para mejorar el UML.
24/10/2026	Pruebas de implementación y ejecución en Visual Studio Code mediante Live Share; estudio de la lógica y creación de un menú de prueba.	Equipo de trabajo.	Pruebas iniciales y revisión de la organización de la lógica.
25/10/2026	Análisis del UML anterior y elaboración de una nueva versión tomando el diseño previo como referencia.	Equipo de trabajo.	Nuevo diseño UML.
26/10/2026	Pruebas de código basadas en el nuevo UML y práctica de explicación del diseño con dos personas externas al equipo.	Equipo de trabajo y dos personas externas.	Revisión de la lógica de los métodos y práctica de explicación.
27/10/2026	Avance del documento escrito: portada, introducción y definición del problema; preparación del UML para la reunión con el profesor.	Equipo de trabajo.	Avance de la documentación y material para revisión.
28/10/2026	Presentación del UML al profesor, análisis de sus recomendaciones, identificación de cambios y continuación del documento escrito.	Equipo de trabajo.	Observaciones del profesor y correcciones por atender.
29/10/2026	Finalización del documento escrito y realización de prácticas para preparar la exposición.	Equipo de trabajo.	Documento escrito finalizado y ensayo de exposición.

30/10/2026	Exposición del UML en la sede Nicoya de la Universidad Nacional.	Equipo de trabajo.	Presentación del diseño UML.

Distribuciones de responsabilidades.

Gerson Andrés Gómez García: Apoyo en el manejo del repositorio Git y orientación al equipo sobre commits, pull y push; colaboración en el diseño, la implementación y la revisión.

Kendal Xavier Obando Mayorga: Participación en el diseño UML, la estructura de clases, las pruebas y la documentación.

Brandon Arturo Gutiérrez Centeno: Participación en el diseño UML, la implementación, la revisión de errores y las pruebas.

Josué Alonso Díaz Calvo: Participación en el diseño UML, la implementación, la documentación y la preparación de la exposición.

Wender Gabriel Villegas García: Participación en el diseño UML, la implementación, la documentación y la preparación de la exposición.

Secuencia de desarrollo del proyecto

1. **Análisis y diseño UML**

Definición de las clases, métodos y relaciones del sistema.

2. **Creación del repositorio y estructura de archivos**

Configuración de Git, ramas y archivos .h.

3. **Implementación del código**

Desarrollo de los archivos .cpp tomando como referencia el UML.

4. **Pruebas y correcciones**

Verificación de métodos, lógica del programa y funcionamiento del menú.

5. **Documentación del proyecto**

Elaboración del documento escrito y actualización de la información técnica.

6. **Preparación de la exposición**

Ensayos, distribución de explicaciones y revisión de los materiales.

7. **Presentación**

Exposición del UML y explicación de las decisiones de diseño.

Actividades complementarias

- **Documentación:** elaboración del documento escrito, incluyendo portada, introducción, definición del problema y demás apartados requeridos. Se debe procurar que la documentación coincida con el UML y la implementación.
- **Pruebas:** ejecución del código en Visual Studio Code, comprobación de métodos, revisión de errores y validación de la lógica del programa.
- **Guías y explicaciones**: preparación de explicaciones sobre el funcionamiento de las clases, métodos y relaciones del UML para facilitar la comprensión del proyecto.
- **Repositorio:** utilización de Git para almacenar y organizar los archivos, crear ramas de trabajo y registrar los cambios mediante commits, así como integrar las modificaciones mediante pull y push.
- **Preparación de las exposiciones**: realización de ensayos grupales, distribución de los temas entre los integrantes y práctica de respuestas a posibles preguntas del profesor.
- **Revisión final:** análisis de las observaciones recibidas por parte de la tutora y el profesor, identificación de errores y aplicación de las correcciones correspondientes.

Control del proyecto

El seguimiento se realiza mediante reuniones presenciales y virtuales, utilizando Discord y sesiones colaborativas en Visual Studio Code con Live Share. En estas reuniones se revisan los avances, se identifican dificultades y se acuerdan las modificaciones necesarias.

Asimismo, se utiliza el repositorio Git como herramienta de control de versiones, permitiendo mantener organizados los cambios realizados por los integrantes

Minutas

Reunión 1

1. Datos de la reunión

Fecha: 20 de octubre de 2026.

Lugar o medio: Plataforma virtual Discord, en dos sesiones: una durante la mañana y otra durante la noche.

2. Integrantes presentes

- Gerson Gómez García: presente durante la mañana y la noche.
- Kendal Xavier Obando Mayorga: presente durante la mañana y la noche.
- Brandon Arturo Gutiérrez Centeno: presente durante la noche.
- Wender Gabriel Villegas García: presente durante la noche.
- Josué Alonso Díaz Calvo: ausente por motivos familiares.

3. Actividades realizadas

Se comenzó a realizar el UML en Lucid chart. Anteriormente, se había pensado cómo realizarlo e implementarlo, pero únicamente se contaba con ideas. Además, se creó un repositorio Git de prueba con el propósito de practicar su funcionamiento, específicamente los comandos pull, push y commit.

Reunión 2

1. Datos de la reunión

Fecha: 21 de octubre de 2026.

Lugar o medio: Presencial, en dos sesiones: una de 8:00 a. m. a 10:00 a. m. y otra de 1:00 p. m. a 3:00 p. m. También se realizó una sesión virtual con una duración de 1 hora y 17 minutos.

2. Integrantes presentes

- Gerson Gómez García: presente en todas las sesiones.
- Kendal Xavier Obando Mayorga: presente en todas las sesiones.
- Brandon Arturo Gutiérrez Centeno: presente en las sesiones de la tarde y la noche.
- Wender Gabriel Villegas García: presente en las sesiones de la mañana y la noche.
- Josué Alonso Díaz Calvo: presente en todas las sesiones.

3. Actividades realizadas

En la sesión de la mañana, nos reunimos con la tutora, quien revisó nuestro UML y nos brindó recomendaciones y aspectos por corregir, los cuales atendimos. En la sesión de la tarde, continuamos con las correcciones e implementamos las clases y los métodos que consideramos necesarios para nuestro UML, con el propósito de desarrollar un código excelente y eficiente.

En la sesión de la noche, continuamos trabajando en el UML y, además, creamos el repositorio Git de nuestro proyecto Black-Jack-UNA. Se creó la rama feature/esqueletoClases y se implementaron algunos archivos .h en dicha rama.

Reunión 3

1. Datos de la reunión

Fecha: 22 de octubre de 2026.

Lugar o medio: Plataforma virtual Discord.

2. Integrantes presentes

- Gerson Gómez García.
- Kendal Xavier Obando Mayorga.
- Brandon Arturo Gutiérrez Centeno.
- Wender Gabriel Villegas García.
- Josué Alonso Díaz Calvo.

3. Actividades realizadas

Gran parte de la reunión se dedicó a discutir cómo nos dividiríamos el trabajo. Gerson nos enseñó aspectos de Git que desconocíamos y se terminó de implementar todos los archivos .h.

Reunión 4

1. Datos de la reunión

Fecha: 23 de octubre de 2026.

Lugar o medio: Presencial.

2. Integrantes presentes

- Gerson Gómez García.
- Kendal Xavier Obando Mayorga.
- Brandon Arturo Gutiérrez Centeno.
- Wender Gabriel Villegas García.
- Josué Alonso Díaz Calvo.

3. Actividades realizadas

Se comenzó a trabajar en los archivos .cpp correspondientes a cada integrante; sin embargo, se detectaron errores en algunos archivos .h, por lo que se suspendió temporalmente el trabajo en los .cpp para solucionar dichos errores.

Durante la noche, el profesor brindó un espacio para que cada grupo pudiera realizar preguntas sobre el proyecto. Por este motivo, nuestro grupo le planteó las dudas existentes. Además, el profesor revisó y analizó nuestro UML, y nos brindó recomendaciones y aspectos por corregir.

Reunión 5

1. Datos de la reunión

Fecha: 24 de octubre de 2026.

Lugar o medio: Virtual.

2. Integrantes presentes

- Gerson Gómez García.
- Kendal Xavier Obando Mayorga.
- Brandon Arturo Gutiérrez Centeno.
- Wender Gabriel Villegas García.
- Josué Alonso Díaz Calvo.

3. Actividades realizadas

Se realizaron pruebas de implementación y ejecución de código en Visual Studio Code mediante Live Share, con el objetivo de trabajar de manera colaborativa. Se estudió la lógica del programa, se creó un menú de prueba y se continuó con el planteamiento del UML. Además, se trabajó en la reorganización de la lógica del programa para mantener una estructura coherente con el diseño establecido.

Reunión 6

1. Datos de la reunión

Fecha: 25 de octubre de 2026.

Lugar o medio: Virtual.

2. Integrantes presentes

- Gerson Gómez García.
- Kendal Xavier Obando Mayorga.
- Brandon Arturo Gutiérrez Centeno.
- Wender Gabriel Villegas García.
- Josué Alonso Díaz Calvo.

3. Actividades realizadas

Se realizó un análisis exhaustivo del UML y se decidió crear uno nuevo tomando como base el diseño anterior. Toda la sesión se dedicó a la elaboración del nuevo UML, utilizando el diseño previo como referencia para realizar las modificaciones necesarias.

Reunión 7

1. Datos de la reunión

Fecha: 26 de octubre de 2026.

Lugar o medio: Virtual.

2. Integrantes presentes

- Gerson Gómez García.
- Kendal Xavier Obando Mayorga.
- Brandon Arturo Gutiérrez Centeno.
- Wender Gabriel Villegas García.
- Josué Alonso Díaz Calvo.

3. Actividades realizadas

Una vez elaborado el nuevo UML, se realizaron pruebas de código basadas en su estructura para comprobar la lógica establecida en cada uno de los métodos. Además, se contó con la participación de dos personas ajenas al grupo, con el objetivo de conocer sus opiniones y practicar la forma en que explicábamos el funcionamiento del UML, así como su relación con situaciones de la vida real.

Reunión 8

1. Datos de la reunión

Fecha: 27 de octubre de 2026.

Lugar o medio: Virtual.

2. Integrantes presentes

- Gerson Gómez García.
- Kendal Xavier Obando Mayorga.
- Brandon Arturo Gutiérrez Centeno.
- Wender Gabriel Villegas García.
- Josué Alonso Díaz Calvo.

3. Actividades realizadas

Se dedicó un tiempo considerable al desarrollo del documento escrito, procurando mantener una estructura ordenada y una presentación formal. Además, se dejó preparado el UML para la reunión programada para el día siguiente mediante Microsoft Teams con el profesor, con el propósito de evacuar las dudas existentes, identificar posibles errores y realizar las correcciones necesarias para mejorar el proyecto.

En cuanto al documento escrito, se trabajó en la elaboración de la portada, la introducción y la definición del problema, procurando establecer una base clara y coherente para el desarrollo del proyecto.

Reunión 9

1. Datos de la reunión

Fecha: 28 de octubre de 2026.

Lugar o medio: Biblioteca de la Universidad Nacional, Sede Nicoya.

2. Integrantes presentes

- Gerson Gómez García.
- Kendal Xavier Obando Mayorga.
- Brandon Arturo Gutiérrez Centeno.
- Wender Gabriel Villegas García.
- Josué Alonso Díaz Calvo.

3. Actividades realizadas

Se realizó la reunión con el profesor, durante la cual se le presentó el UML del proyecto. El profesor brindó consejos, comentarios y recomendaciones que nos permitieron obtener una nueva perspectiva sobre el diseño y la lógica establecida. Asimismo, se evacuaron las dudas existentes para comprender mejor los aspectos que requerían atención.

Una vez finalizada la reunión, nos reunimos como equipo de trabajo para analizar las observaciones del profesor, discutir los cambios necesarios y comenzar a solucionar los errores identificados. Además, se continuó con la elaboración del documento escrito, avanzando en su desarrollo y procurando mantener la coherencia entre la documentación y el diseño del proyecto.

Reunión 10

1. Datos de la reunión

Fecha: 28 de octubre de 2026.

Lugar o medio: Biblioteca de la Universidad Nacional, Sede Nicoya.

2. Integrantes presentes

- Gerson Gómez García.
- Kendal Xavier Obando Mayorga.
- Brandon Arturo Gutiérrez Centeno.
- Wender Gabriel Villegas García.
- Josué Alonso Díaz Calvo.

3. Actividades realizadas

Se realizó la reunión con el profesor, durante la cual se le presentó el UML del proyecto. El profesor brindó consejos, comentarios y recomendaciones que nos permitieron obtener una nueva perspectiva sobre el diseño y la lógica establecida. Asimismo, se evacuaron las dudas existentes para comprender mejor los aspectos que requerían atención.

Una vez finalizada la reunión, nos reunimos como equipo de trabajo para analizar las observaciones del profesor, discutir los cambios necesarios y comenzar a solucionar los errores identificados. Además, se continuó con la elaboración del documento escrito, avanzando en su desarrollo y procurando mantener la coherencia entre la documentación y el diseño del proyecto.

Reunión 11

1. Datos de la reunión

Fecha: 29 de octubre de 2026.

Lugar o medio: Virtual.

2. Integrantes presentes

- Gerson Gómez García.
- Kendal Xavier Obando Mayorga.
- Brandon Arturo Gutiérrez Centeno.
- Wender Gabriel Villegas García.
- Josué Alonso Díaz Calvo.

3. Actividades realizadas

Se finalizó la elaboración del documento escrito y se realizaron diversas pruebas de práctica con el propósito de prepararnos adecuadamente para el día de la exposición. Durante estas sesiones, se repasaron los aspectos más importantes del proyecto, se practicaron las explicaciones y se pulieron varios detalles para mejorar la presentación,

fortalecer el dominio del tema y procurar que todos los integrantes estuvieran preparados para exponer de manera clara, ordenada y segura.

Reunión 12

1. Datos de la reunión

Fecha: 30 de octubre de 2026.

Lugar o medio: Universidad Nacional, Sede Nicoya.

2. Integrantes presentes

- Gerson Gómez García.
- Kendal Xavier Obando Mayorga.
- Brandon Arturo Gutiérrez Centeno.
- Wender Gabriel Villegas García.
- Josué Alonso Díaz Calvo.

3. Actividades realizadas

Se realizó la exposición del UML correspondiente al proyecto Black-Jack-UNA.

Avance de la implementación

Clase implementada: Casa

La clase Casa representa a la banca del juego y se encarga de administrar su dinero y de validar las apuestas. Sus atributos son privados: el nombre de la casa ("Casa"), el dinero disponible en la banca (2000), la apuesta mínima (10) y la apuesta máxima (500). Estos valores se asignan por defecto, por lo que el constructor Casa() no necesita recibir parámetros.

Funcionalidades implementadas

- **apuestaValida(int monto):** devuelve true solo si el monto está entre la apuesta mínima y la máxima, y además la banca tiene dinero suficiente para cubrirlo. Así se evita aceptar apuestas que la casa no podría pagar.
- **retirar(int monto):** representa a la banca pagando un premio. Si el monto es menor o igual a cero, o mayor al dinero disponible, no hace nada y devuelve false. Si es válido, descuenta el monto y devuelve true.
- **depositar(int monto):** representa a la banca cobrando una apuesta perdida. Suma el monto al dinero de la banca solo si es positivo.
- **getDineroBanca():** devuelve el dinero actual de la banca, para mostrarlo o consultarlo desde otras clases.

Casa.cpp > apuestaValida(int)

```cpp
#include "Casa.h"

Casa::Casa() {
}

bool Casa :: apuestaValida(int monto){

    return monto >= apuestaMinima &&
           monto <= apuestaMaxima &&
           monto <= dineroBanca;
}

bool Casa :: retirar(int monto){

    if(monto <= 0 || monto > dineroBanca){
        return false;
    }

    dineroBanca -= monto;
    return true;
}

void Casa :: depositar(int monto){

    if(monto > 0){
        dineroBanca += monto;
    }
}

int Casa :: getDineroBanca(){

    return dineroBanca;
}
```

Casa.h > Casa

```h
#pragma once
#include <iostream>
#include <string>

using namespace std;

class Casa{

private:

string nombre = "Casa";
int dineroBanca = 2000;
int apuestaMinima = 10;
int apuestaMaxima = 500;

public:

Casa();

bool apuestaValida(int monto);

bool retirar(int monto);

void depositar(int monto);

int getDineroBanca();


};
```

Clase implementada: Jugador

La clase Jugador representa a una persona que participa en la partida. Hereda de Participante, por lo que ya cuenta con nombre y mano de cartas, y agrega lo propio del jugador: sus fichas y su apuesta actual. Sus atributos privados son un vector de fichas (fichas) y un entero con la apuesta de la ronda (apuestaActual), que empieza en 0 al crear al jugador. El constructor recibe el nombre y lo pasa a la clase base.

Funcionalidades implementadas

- **quiereCarta():** pregunta por consola "Deseas otra carta (S/N)" y lee la respuesta. Devuelve true si el jugador escribe s o S. Es la implementación de la decisión que cada participante toma en su turno.
- **apostar(int monto):** suma el valor de todas las fichas del jugador y comprueba que el monto sea mayor que cero y no supere ese total. Si es válido, guarda el monto en apuestaActual y devuelve true; si no, devuelve false.
- **recibirFichas(int monto):** convierte un monto de dinero en fichas, de mayor a menor denominación: 100 (negro), 25 (verde), 10 (azul), 5 (rojo) y 1 (blanco), y las agrega al vector. Se usa para entregar premios o fichas iniciales.
- **entregarFichas():** retira la apuesta del jugador cuando pierde. Calcula el total de fichas, vacía el vector, vuelve a repartir en fichas el dinero restante (total menos apuesta), pone la apuesta en 0 y devuelve el monto apostado para que la banca lo cobre.
- **getApuesta():** devuelve la apuesta actual.

Jugador.cpp > ...

```cpp
#include "Jugador.h"
#include "Ficha.h"
using namespace std;

Jugador::Jugador(string nombre) : Participante(nombre) {
    this->apuestaActual = 0;
}

bool Jugador::quiereCarta() {
    char respuesta;
    cout << "Deseas otra carta (S/N)" << endl;
    cin >> respuesta;
    return (respuesta == 's' || respuesta == 'S');
} // fin de quiereCarta

bool Jugador::apostar(int monto) {
    int totalFichas = 0;

    for (Ficha ficha : fichas) {
        totalFichas += ficha.getValor();
    }

    if (monto > 0 && monto <= totalFichas) {
        apuestaActual = monto;
        return true;
    }

    return false;
} // fin de apostar

void Jugador::recibirFichas(int monto) {
    // Denominaciones de mayor a menor, con su color
    const int valores[] = {100, 25, 10, 5, 1};
    const string colores[] = {"negro", "verde", "azul", "rojo", "blanco"};

    for (int i = 0; i < 5; i++) {
        while (monto >= valores[i]) {
            fichas.push_back(Ficha(valores[i], colores[i]));
            monto -= valores[i];
        }
    }
} // fin de recibirFichas

int Jugador::entregarFichas() {
    int apuesta = apuestaActual;
    int totalFichas = 0;

    for (Ficha ficha : fichas) {
        totalFichas += ficha.getValor();
    }

    fichas.clear();
    recibirFichas(totalFichas - apuesta);

    apuestaActual = 0;
    return apuesta;
} // fin de entregarFichas

int Jugador::getApuesta() {
    return apuestaActual;
```

```cpp
#pragma once
#include <iostream>
#include <vector>
#include "Ficha.h"
#include "Participante.h"

using namespace std;

class Jugador : public Participante{

private:

vector<Ficha> fichas;

int apuestaActual;

public:

Jugador(string nombre);

bool quiereCarta();

bool apostar(int monto);

void recibirFichas(int monto);

int entregarFichas();

int getApuesta();

};
```

Clase implementada: Mano

La clase Mano representa el conjunto de cartas que tiene un participante en la ronda. Sus atributos privados son un vector de cartas (cartas) y un entero valorApuesta, que guarda la apuesta asociada a esa mano y empieza en 0. La clase Juego está declarada como friend, por lo que puede acceder directamente a esos atributos.

Funcionalidades implementadas

- **agregarCarta(const Carta& c):** añade una carta a la mano.
- **calcularValor() const:** suma el valor de todas las cartas y cuenta los ases. Como el As vale 11 por defecto, si el total supera 21 se le resta 10 por cada As, hasta que la mano deje de pasarse o no queden ases por ajustar. Así el As vale 1 u 11 según convenga.
- **esBlackJack() const:** devuelve true solo si la mano tiene exactamente dos cartas y suman 21.
- **sePasa() const:** devuelve true si el valor de la mano es mayor que 21.
- **cantidadCartas() const:** devuelve cuántas cartas hay en la mano.
- **limpiar():** elimina todas las cartas y reinicia valorApuesta a 0, dejando la mano lista para la siguiente ronda.
- **getValorApuesta() const:** devuelve la apuesta de la mano.
- **aTexto(bool ocultarPrimera) const:** devuelve la mano como texto, con las cartas separadas por comas. Si ocultarPrimera es true, la primera carta aparece como “[Carta oculta]”, lo que permite esconder una carta del crupier mientras juega el jugador.

Mano.cpp > calcularValor() const

```cpp
#include "Mano.h"

void Mano :: agregarCarta(const Carta& c){
    cartas.push_back(c);
}

int Mano :: calcularValor() const{
    int valor = 0;
    int ases = 0;
    for(const Carta& c : cartas){
     valor+= c.getValor();

     if (c.esAs()){
     ases++;
     }
//El As puede valer 1 o 11 en blackjack.s
//Se reduce su valor en 10 si nos pasamos de 21.
     while(valor > 21 && ases > 0){
     valor -= 10;
     ases--;
     }
     }
     return valor;
}

bool Mano :: esBlackJack() const {
    return cartas.size() == 2 && calcularValor() == 21;

}

bool Mano :: sePasa() const{
    return calcularValor() > 21;
}

int Mano :: cantidadCartas() const{
    return static_cast<int>(cartas.size());
}

void Mano :: limpiar(){
    cartas.clear();
    valorApuesta = 0;
}

int Mano :: getValorApuesta() const {
    return valorApuesta;
}

string Mano :: aTexto(bool ocultarPrimera) const {
    string resultado;

    for(int i = 0; i<cantidadCartas(); i++){
        if(i == 0 && ocultarPrimera){
            resultado += "[Carta oculta]";
        }else{
            //Ajustar segun el metodo que tenga la clase carta
            resultado += cartas[i].mostrarCarta();
        }


        if(i < cantidadCartas()-1){
            resultado+= ",  ";
        }

    }
    return resultado;
}

```

Mano.h > ...

```cpp
#pragma once
#include <iostream>
#include <vector>
#include "Carta.h"
#include <string>

using namespace std;

class Mano{

private:

vector<Carta> cartas;
int valorApuesta = 0;

friend class Juego;

public:

void agregarCarta(const Carta& c);

int calcularValor() const;

bool esBlackJack() const;

bool sePasa() const;

int cantidadCartas() const;

void limpiar();

int getValorApuesta() const;

string aTexto(bool ocultarPrimera) const;

};
```

Clase implementada: Mazo

La clase Mazo representa la baraja con la que se juega. Su único atributo privado es un vector de cartas (cartas). El mazo se construye con una baraja francesa estándar de 52 cartas (4 palos: Corazones, Diamantes, Tréboles y Picas, con números del 1 al 13) y la mezcla de forma aleatoria al crearse. Para mezclar usa un generador mt19937 inicializado con la hora del sistema, de modo que el orden cambia en cada ejecución.

Funcionalidades implementadas

- **llenar() (privado):** vacía el vector, crea las 52 cartas y las baraja. El constructor Mazo() simplemente lo invoca.
- **sacarCarta():** retira y devuelve la última carta del mazo. Si el mazo está vacío, lo vuelve a llenar y barajar antes de sacarla, por lo que nunca se queda sin cartas.
- **estaVacio():** devuelve true si no quedan cartas.
- **cartasRestantes():** devuelve cuántas cartas quedan.
- **necesitaBarajar():** indica si hay que volver a barajar; en esta versión devuelve true solo cuando el mazo está vacío.

Mazo.h > Mazo > llenar()

```h
#pragma once
#include <iostream>
#include <vector>
#include "Carta.h"

using namespace std;

class Mazo {

private:
    vector<Carta> cartas;
    void llenar();

public:

    Mazo();

    Carta sacarCarta();

    bool estaVacio();

    int cartasRestantes();

    bool necesitaBarajar();

};
```

Mazo.cpp > llenar()

```cpp
#include "Mazo.h"
#include <algorithm>
#include <random>
#include <chrono>

void Mazo::llenar() {
    cartas.clear();

    string palos[4] = { "Corazones", "Diamantes", "Treboles", "Picas" };

    for (int p = 0; p < 4; p++) {
        for (int numero = 1; numero <= 13; numero++) {
            cartas.push_back(Carta(palos[p], numero));
        }
    }

    static mt19937 generador(
    static_cast<mt19937::result_type>(
        chrono::high_resolution_clock::now().time_since_epoch().count()
    )
);

shuffle(cartas.begin(), cartas.end(), generador);
}

Mazo::Mazo() {
    llenar();
}

Carta Mazo::sacarCarta() {

    if (cartas.empty()) {
        llenar();
    }

    Carta carta = cartas.back();
    cartas.pop_back();
    return carta;
}

bool Mazo::estaVacio() {
    return cartas.empty();
}

int Mazo::cartasRestantes() {
    return (int)cartas.size();
}


bool Mazo::necesitaBarajar() {
    return cartas.empty();
}
```

Clase implementada: Participante

La clase Participante es la clase base de quienes juegan una ronda: Jugador y Crupier heredan de ella. Guarda lo que tienen en común: el nombre (nombre) y un vector de manos (manos), de modo que en el futuro un participante pueda tener más de una mano. Sus atributos son protegidos, para que las clases hijas accedan a ellos directamente. Tiene un destructor virtual y declara a Juego como friend.

Funcionalidades implementadas

- **Participante(string nombre):** inicializa el nombre del participante.
- **quiereCarta():** pregunta por consola "¿deseas pedir otra carta? (s/n)" y devuelve true si la respuesta es s o S. Es el comportamiento por defecto, que cada clase hija puede reemplazar con su propia lógica.
- **recibirCarta(const Carta& c):** agrega una carta a la mano. Si el participante aún no tiene ninguna mano, crea una antes de agregarla.
- **getMano() const:** devuelve una referencia de solo lectura a la mano principal (la primera del vector).
- **limpiarMano():** elimina las manos del participante para empezar una nueva ronda.
- **getNombre() const:** devuelve el nombre.

Participante.h > ...

```cpp
#pragma once
#include <iostream>
#include <vector>
#include "Mano.h"
#include "Carta.h"

using namespace std;

class Participante{

protected:

string nombre;

vector<Mano>manos;

friend class Juego;

public:

Participante(string nombre);

virtual ~Participante();

bool quiereCarta();

void recibirCarta(const Carta& c);

const Mano& getMano() const;

void limpiarMano();

string getNombre() const;

};
```

Participante.cpp > getMano() const

```cpp
#include "Participante.h"

Participante :: Participante(string nombre)
:nombre(nombre){
}

Participante :: ~Participante(){}

bool Participante :: quiereCarta(){
    char opcion;

    cout << nombre << ", ¿deseas pedir otra carta? (s/n): ";
    cin >> opcion;

    return opcion == 's' || opcion == 'S';
}

void Participante :: recibirCarta(const Carta& c){
    if (manos.empty()) {
        manos.push_back(Mano());
    }

    manos.back().agregarCarta(c);
}

const Mano & Participante::getMano() const{
    return manos.front();
}

void Participante :: limpiarMano(){
    manos.clear();
}

string Participante :: getNombre() const {
    return nombre;
}
```